Informe breve de diagnóstico y mejora

De comprender el código a transformar la experiencia

Actividad: Rediseño de una pantalla de Hotel Cartagena con Flutter
Pantalla asignada: `lib/screens/rooms_screen.dart` — `RoomsScreen` (RF03: consultar habitaciones disponibles)
Asignatura: Electiva II · Dart y Flutter

---

1. Propósito de la pantalla

`RoomsScreen` permite al recepcionista **ver qué habitaciones están disponibles** y **elegir una para reservar**. Agrupa las habitaciones disponibles por tipo (Sencilla, Doble, Suite) y muestra su capacidad. Es el punto de partida natural para la acción de reservar (RF04/RF05).

---

2. Diagnóstico de la versión original

Se analizó el código original (`rooms_screen.dart` , versión del repositorio base) y se identificaron **6 oportunidades de mejora**.

#	Elemento observado	Problema u oportunidad	Consecuencia para el usuario	Evidencia
P1	Chip de estado (`_EstadoChip`)	El chip siempre dice "Disponible" en verde	Ruido visual: ocupa espacio sin aportar información	`_EstadoChip` (líneas 75–92); solo se listan disponibles (`consultarDisponiblesAgrupadas()`)
P2	Filas de habitación (`ListTile`)	No llevan a ninguna acción	Para reservar hay que volver al menú, entrar a Reservar y escribir	`ListTile` sin `onTap` ni botones (líneas 53–59)

			el número de memoria: riesgo de error y pérdida de tiempo	
P3	Capacidad	No ayuda a elegir según el número de huéspedes	Con un grupo de 3, el recepcionista debe deducir que solo sirve una Suite	La capacidad solo aparece como texto en el `subtitle` (líneas 56–57)
P4	Listado	No hay panorama general del hotel	Con "0 disponibles" no se sabe si el hotel está lleno o qué está reservado/ocupado	Solo se muestran disponibles; no hay totales (`hotel.dart` , `consultarDisponiblesAgrupadas`)
P5	Estado de la pantalla	Los datos pueden quedar desactualizados	Si se vuelve a la pantalla desde otra, no se refresca y no hay forma de actualizar	`RoomsScreen` es `StatelessWidget` , sin `RefreshIndicator` (línea 14)
P6	Textos	Textos poco naturales y estado vacío sin orientación	Lectura menos clara ("disponible(s)", "persona(s)"); el aviso vacío no sugiere qué hacer	Cadenas en el `title` del `ExpansionTile` (línea 42) y en el `ListTile` de aviso (línea 49)

¿Por qué merecen atención? Cada una afecta directamente la tarea del usuario (encontrar y reservar rápido, sin equivocarse), que es el propósito de la pantalla.

---

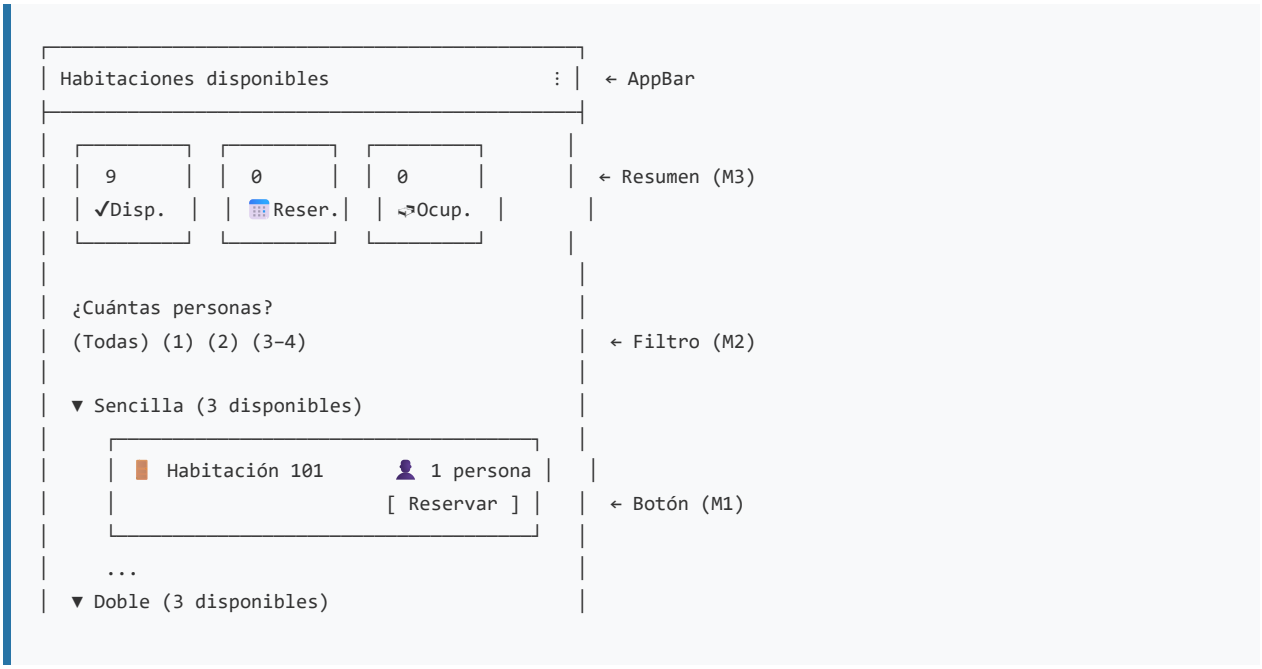
3. Mejoras priorizadas

#	Problema que resuelve	Mejora propuesta	Beneficio esperado	Forma de comprobarla

M1	P2	Botón **Reservar** en cada habitación que abre `ReserveScreen` con el número precargado	Menos pasos y cero errores al digitar el número	Reservar desde la lista sin escribir el número; contar toques antes/después
M2	P3	Filtro **"¿Cuántas personas?"** (1 · 2 · 3–4)	Ver solo habitaciones que sirven al huésped	Seleccionar "3–4" y verificar que solo aparecen Suites
M3	P4	**Resumen superior** con disponibles, reservadas y ocupadas	Panorama del hotel de un vistazo	Reservar/check-in y verificar que los contadores cambian
M4	P5	**Actualización automática y manual** (`StatefulWidget` + `RefreshIndicator`)	La lista siempre refleja el estado real	Reservar desde la lista y comprobar que la habitación desaparece al volver
M5	P1 y P6	**Limpieza visual y textos**	Pantalla más limpia y fácil de leer	Comparar capturas antes/después; probar en pantalla pequeña sin desbordes

---

4. Propuesta de diseño (boceto)



```
| ▼ Suite (3 disponibles)                    |
└────────────────────────────────────────────┘
```

Decisiones de diseño:

- **Jerarquía:** el resumen va arriba (información de contexto) y el filtro justo debajo, antes de la lista.
- **Acción principal:** el botón "Reservar" queda a la derecha de cada fila, con estilo `FilledButton.tonal` para no competir con el botón principal de confirmación de `ReserveScreen`.
- **Estados de interacción:** el filtro resalta la opción elegida (`ChoiceChip.selected`); al volver de la reserva la lista se reconstruye; `RefreshIndicator` permite actualizar manualmente.
- **Estado vacío:** si no hay resultados, se muestra un mensaje con sugerencia ("Ajusta el filtro...").
- **Adaptabilidad:** el resumen usa `Row` + `Expanded` (3 tarjetas que se reparten el ancho) y la lista usa `ListView` con `padding`, para evitar desbordes en pantallas angostas.

---

5. Implementación técnica

5.1 `rooms_screen.dart` pasa a `StatefulWidget`

Permite guardar el filtro seleccionado y refrescar la vista.

```
class RoomsScreen extends StatefulWidget {
  final Hotel hotel;
  const RoomsScreen({super.key, required this.hotel});
  @override
  State<RoomsScreen> createState() => _RoomsScreenState();
}
```

5.2 Filtro "¿Cuántas personas?" (M2)

Solo afecta la presentación; **no valida nada** en `Hotel`.

```
static const _filtros = [
  _FiltroPersonas('Todas', 1),
  _FiltroPersonas('1', 1),
  _FiltroPersonas('2', 2),
```

```
  _FiltroPersonas('3-4', 3),
];
int _filtroSeleccionado = 0;

bool _cumpleFiltro(Habitacion h, int minimo) => h.capacidad >= minimo;
```

5.3 Resumen superior (M3)

Lee `hotel.habitaciones` (solo lectura) y cuenta por estado.

```
final disponibles = hotel.habitaciones.where((h) => h.estaDisponible).length;
final reservadas  = hotel.habitaciones.where((h) => h.estaReservada).length;
final ocupadas    = hotel.habitaciones.where((h) => h.estaOcupada).length;
```

5.4 Botón "Reservar" y actualización al volver (M1 + M4)

```
Future<void> _irAReservar(Habitacion habitacion) async {
  await Navigator.of(context).push(
    MaterialPageRoute(
      builder: (_) => ReserveScreen(
        hotel: widget.hotel,
        numeroInicial: habitacion.numero, // precarga el número
      ),
    ),
  );
  if (mounted) setState(() {}); // reconstruye: la habitación ya reservada desaparece
}
```

5.5 `RefreshIndicator` (M4)

```
body: RefreshIndicator(
  onRefresh: _refrescar,
  child: ListView(padding: const EdgeInsets.all(16), children: [ ... ]),
),
```

5.6 Cambio mínimo fuera de la pantalla (M1)

`ReserveScreen` recibe un parámetro opcional; si no se envía, se comporta igual que antes.

```
const ReserveScreen({super.key, required this.hotel, this.numeroInicial});

@override
void initState() {
  super.initState();
```

```
    if (widget.numeroInicial != null) {
      _numeroHabitacionController.text = '${widget.numeroInicial}';
    }
  }
```

5.7 Limpieza visual (M5)

- Se eliminó `_EstadoChip`.
- Capacidad con ícono `Icons.person` y plural correcto (`1 persona` / `N personas`).
- Estado vacío con sugerencia.

5.8 Reglas de negocio conservadas

No se modificó `lib/services/hotel.dart`. La disponibilidad se sigue validando en `reservarHabitacion()` y la capacidad en `realizarCheckIn()`. El filtro y el resumen son solo de presentación.

---

6. Pruebas realizadas

6.1 Pruebas automatizadas (`flutter test`)

Se agregó `test/rooms_screen_test.dart` con 4 pruebas de widget. Se ejecutaron con `flutter test` y **todas pasan (5/5, incluida la prueba original de login)**.

#	Prueba	Qué verifica	Resultado
A1	Muestra el resumen y el filtro	Aparecen "Disponibles / Reservadas / Ocupadas", el filtro "¿Cuántas personas?" y los 9 botones "Reservar"	**Comprobado**
A2	El filtro "3–4" deja solo las Suites	Aparecen 2 avisos "Sin habitaciones disponibles" y 3 botones "Reservar"	**Comprobado**
A3	El botón "Reservar" abre la reserva con el número	Se abre `ReserveScreen` y el campo número contiene "101"	**Comprobado**
A4	El estado vacío sugiere qué hacer	Con las Suites reservadas y filtro "3–4", aparece "No hay habitaciones disponibles" + "Ajusta el filtro…"	**Comprobado**

6.2 Pruebas manuales (por ejecutar en Chrome)

#	Escenario	Acción	Resultado esperado	Resultado obtenido	¿Cumple?
M1	Uso normal	Filtrar por 2 personas y reservar la 104 desde la lista	`ReserveScreen` abre con "104"; al volver, la 104 ya no aparece y el resumen se actualiza	(pendiente)	—
M2	Datos incorrectos	En la reserva precargada, poner 0 días	El formulario y `Hotel` rechazan la reserva con mensaje de error	(pendiente)	—
M3	Error de negocio	Reservar dos veces la misma habitación	`HotelException` : "La habitación X no está disponible."	(pendiente)	—
M4	Otro tamaño de pantalla	Ejecutar en ventana angosta (≈360 px) y ancha	Sin desbordes; contenido legible	(pendiente)	—
M5	Refresco manual	Arrastrar hacia abajo en la lista	La vista se reconstruye sin errores	(pendiente)	—

---

7. Comparación antes / después (trazabilidad)

Problema inicial	Cambio implementado	Evidencia	Conclusión
P2 — sin acción de reservar	Botón "Reservar" por habitación + `numeroInicial` en `ReserveScreen`	Prueba automatizada **A3** (abre con "101") + fragmento de código	**Comprobado**
P3 — no ayuda a elegir por huéspedes	Filtro de personas con `ChoiceChip`	Prueba automatizada **A2** (filtro "3–4" deja 3 Suites)	**Comprobado**
P4 — sin panorama general	Resumen de disponibles/reservadas/ocupadas	Prueba automatizada **A1** (aparecen los 3 contadores)	**Comprobado**

P5 — datos desactualizados	`StatefulWidget` + `setState` al volver + `RefreshIndicator`	Código en `rooms_screen.dart` + prueba manual M1/M5 pendiente	**Observado**
P1/P6 — chip redundante y textos	Se quitó el chip; ícono de persona; plurales; estado vacío con sugerencia	Pruebas automatizadas **A1** y **A4** + comparativa visual	**Comprobado**

---

8. Conclusiones

- La mejora más significativa es **M1**: convierte la pantalla de "solo mirar" en "mirar y actuar", eliminando el paso de volver al menú y escribir el número (origen de errores).
- **M2 y M3** dan al recepcionista el contexto (personas y ocupación) para decidir más rápido.
- **M4** garantiza que la información mostrada sea confiable.
- **M5** reduce el ruido visual y mejora la legibilidad.
- **Aprendizaje:** una mejora de interfaz no requiere tocar la lógica de negocio; separar presentación de reglas permite cambiar la experiencia sin riesgo.
- **Próxima versión:** permitir ordenar por capacidad, mostrar el estado de reservadas/ocupadas también en la lista (con el chip recuperado de forma útil) y agregar una prueba de widget automatizada para el filtro.